Bubble Sort

Given an array of N integers, **sort** the array using the **bubble sort** algorithm.

Example

Input

0	1	2	3	4
50	40	30	20	10

Output

0	1	2	3	4
10	20	30	40	50

- Bubble Sort
- Selection Sort
- Insertion Sort

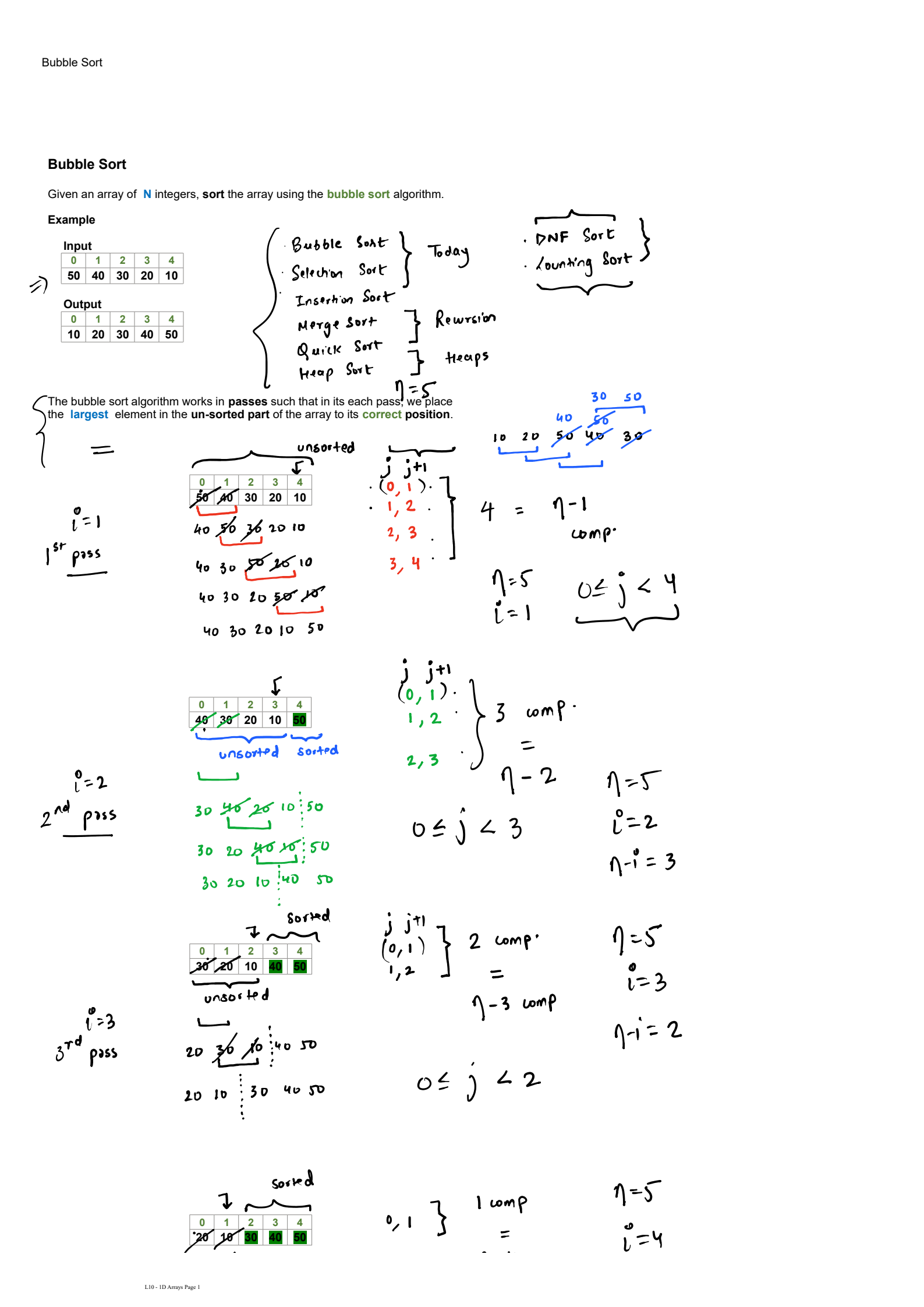
} Today

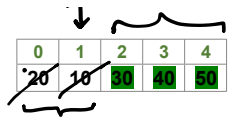
- Merge Sort
- Quick Sort
- Heap Sort

} Recursion / Heaps

- DNF Sort
- Counting Sort

The bubble sort algorithm works in **passes** such that in its each pass, we place the **largest** element in the **un-sorted part** of the array to its **correct position**.





$i = 4$

4th pass

10 20 30 40 50

0, 1 } 1 comp

= $n-4$ comp

$0 \le j < 1$

$i = 4$

$n - i = 1$

for an array of size n, BS will have $n-1$ passes

```
for (int i = 1; i <= n-1; i++) {
    // in the ith pass, place the
    // largest element in unsorted
    // part of arr[] to its corr. pos.
}
```

$n = 5$

$i = 1$ $\quad 4 = n-1$

$i = 2$ $\quad 3 = n-2$

$i = 3$ $\quad 2 = n-3$

$i = 4$ $\quad 1 = n-4$

i^{th} pass

$n - i$ comp'

j , $j+1$

i^{th} pass

$1 \le i \le n-1$

$0 \le j < n-i$

1 to n } n times
0 to n-1 }

```
void bubbleSort(int arr[], int n) {
    for (int i = 1; i <= n - 1; i++) {
        // in the ith, place the largest ele
        // part of the arr[] to its correct
        for (int j = 0; j < n - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

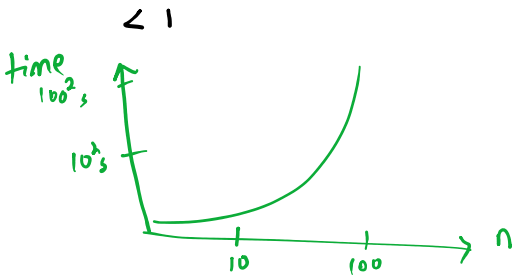
i	j	#comp
1	0 to $< n-1$	$n-1$
2	0 to $< n-2$	$n-2$
3	0 to $< n-3$	$n-3$
⋮	⋮	⋮
$n-1$	0 to $< n-(n-1)$ < 1	1

$\sum = 1 + \dots + n-1 + n - n$

$= \frac{n(n+1)}{2} - n$

$= \frac{n(n-1)}{2}$

time $\propto n^2$ ↑



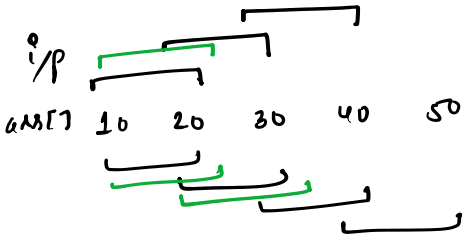
< 1

$\frac{n^2}{2}$

$\sim n^2 \cdot const$

$\therefore O(n^2)$

↳ Big-O



1st pass

2nd pass

3rd pass

4th pass

If array is already sorted then BS will stop after the 1st pass i.e. we'll do only $n-1$ comp $\therefore \Omega(n)$

↳ Big-Omega

Selection Sort

Given an array of **N** integers, **sort** the array using the **selection sort** algorithm.

Example

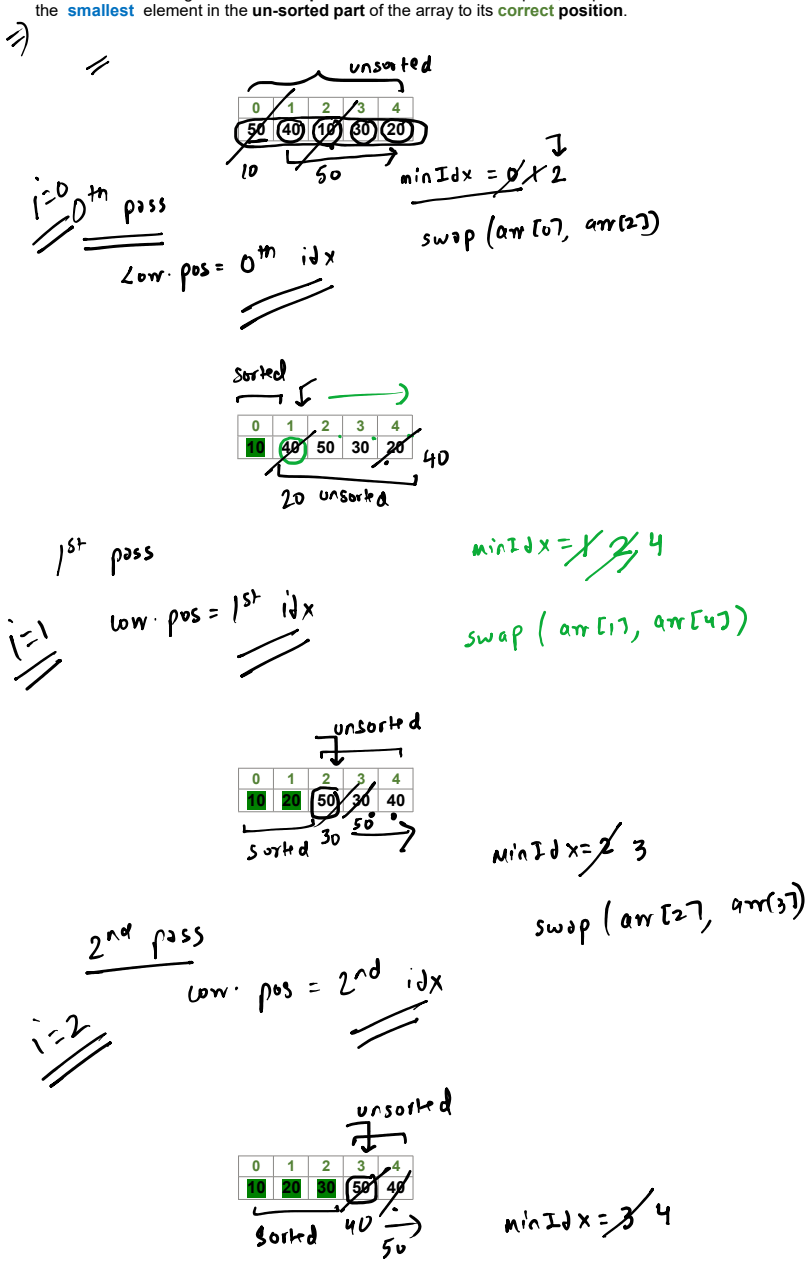
Input

0	1	2	3	4
50	40	10	30	20

Output

0	1	2	3	4
10	20	30	40	50

The selection sort algorithm works in **passes** such that in its each pass, we place the **smallest** element in the **un-sorted part** of the array to its **correct** **position**.



for an array of size n, SS will have $n-1$ passes

```
for (i = 0; i ≤ n-2; i++) {
    // in the pass, place the
    // smallest element in the unsorted
    // part of arr[] to its corr. pos.
    // i.e. ith index
}
```

minIdx = i

idx i

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i <= n - 2; i++) {
        // in the ith pass, place the smallest
        // unsorted part of the arr[] to its co
        // i.e. ith index
        int minIdx = i; // at the start of the
        for (int j = i + 1; j <= n - 1; j++) {
            if (arr[j] < arr[minIdx]) { // you'
                minIdx = j; // therefore update
            }
        }
        swap(arr[minIdx], arr[i]);
    }
}
```

	i	j	# comp
$n-1$ passes	0	1 to $n-1$	$n-1$
	1	2 to $n-1$	$n-2$
	2	3 to $n-1$	$n-3$
	⋮	⋮	⋮
	$n-2$	$n-1$ to $n-1$	1

$$\sum = \frac{n(n-1)}{2} \sim n^2 \cdot \text{const}$$

$O(n^2)$

10 50 40 30 20

0th pass

minIdx = 0

Insertion Sort

Given an array of **N** integers, **sort** the array using the **insertion sort** algorithm.

Example

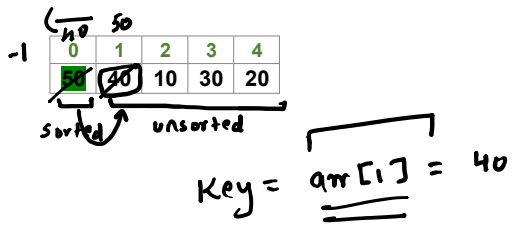
Input

0	1	2	3	4
50	40	10	30	20

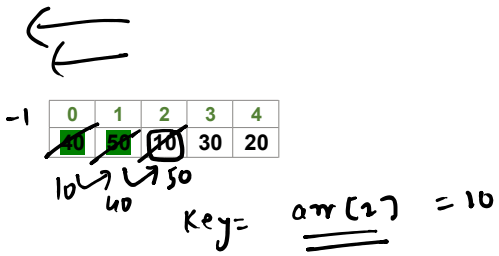
Output

0	1	2	3	4
10	20	30	40	50

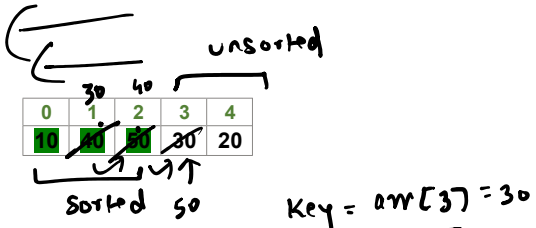
The insertion sort algorithm works in **passes** such that in its each pass, we place the **first** element in the **un-sorted part** of the array to its **correct** **position** in the **sorted** part of the array.



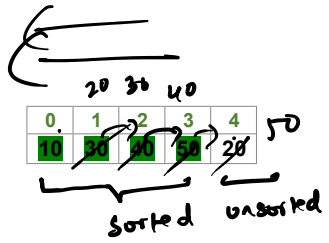
1st pass



2nd pass



3rd pass



4th pass

key = arr[4] = 20

for an array of size n, IS will have n-1 passes

```
for (i=1 ; i<=n-1 ; i++) {
    key = arr[i];
    // insert the key to its
    // corr. pos. in the sorted
    // part of the array
}
```

```
void insertionSort(int arr[], int n) {

    for (int i = 1; i <= n - 1; i++) {

        int key = arr[i];

        // in the ith pass, insert the key
        // in the sorted part of the array

        int j = i - 1;

        while (j >= 0 and key < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;

    }
}
```

i	j	#comp.
1	0 to 0	1
2	1 to 0	2
3	2 to 0	3
⋮	⋮	⋮
n-1	n-2 to 0	n-1

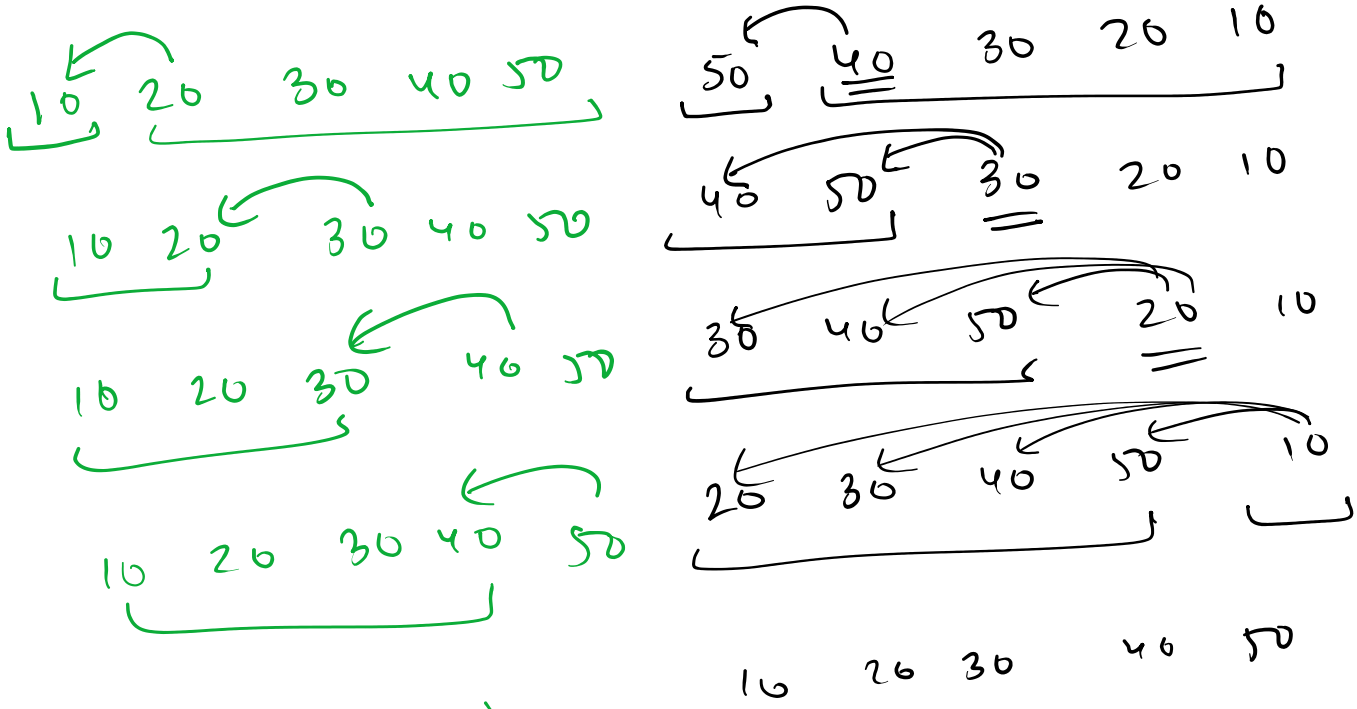
$\sum = \frac{n(n-1)}{2}$

$\sim n^2$: const

$\therefore O(n^2)$

If array is in dec. order then we'll encounter the worst-case

50 40 30 20 10



If your array is already sorted then you'll do only one comp in each pass

∴ total we'll do n-1 comp' across all passes so time is $\Omega(n)$

Binary Search

Given a sorted array of **N** distinct integers & an integer **T**, implement the **binary search** algorithm to find the **index** of T in the given array.

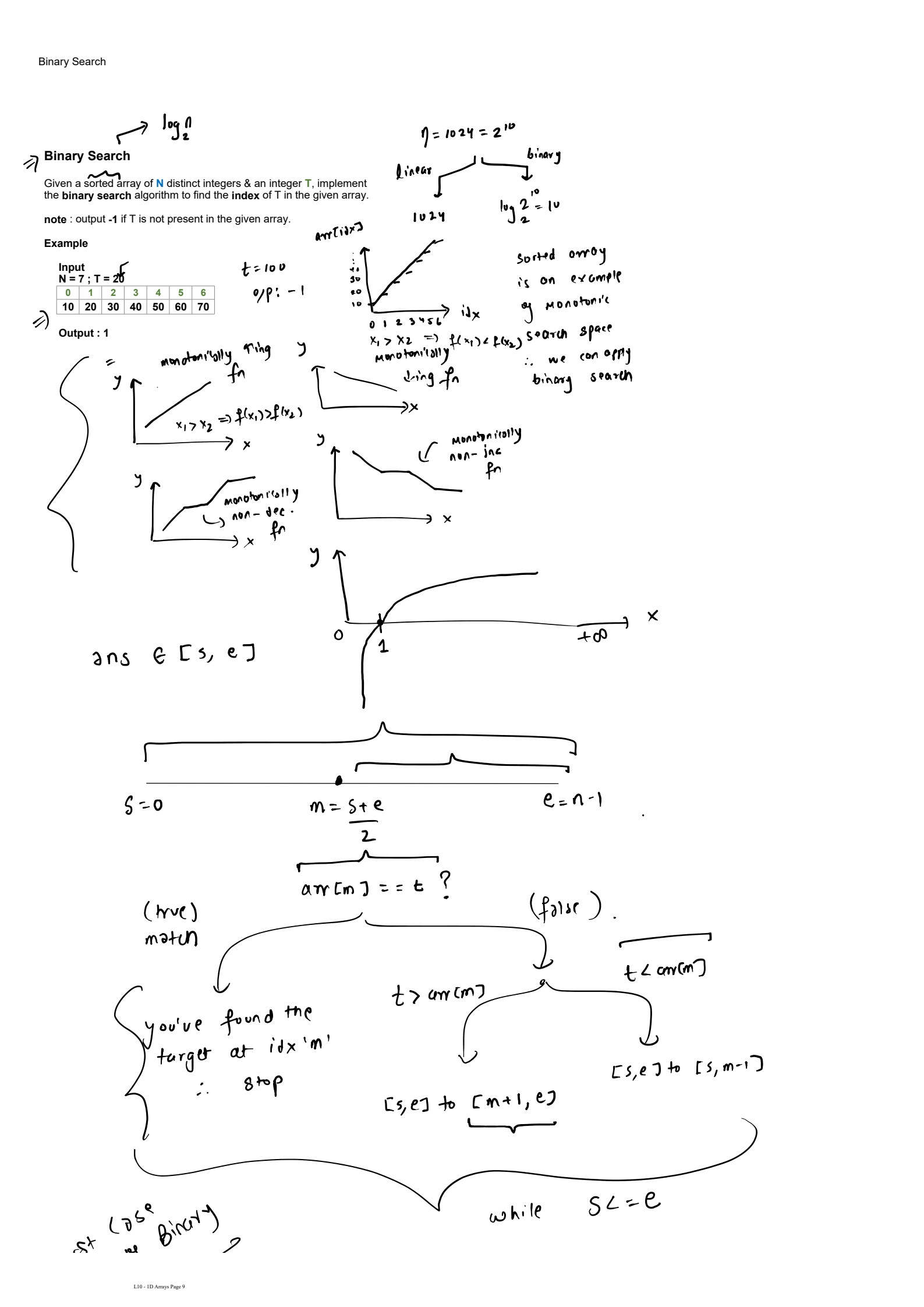
note : output **-1** if T is not present in the given array.

Example

Input
N = 7 ; T = 20

0	1	2	3	4	5	6
10	20	30	40	50	60	70

Output : 1



Worst Case Analysis of Binary Search

while $S <= E$

$n = \frac{n}{2^0} \longrightarrow 1$

$n/2 = \frac{n}{2^1} \longrightarrow 2$

$n/4 = \frac{n}{2^2} \longrightarrow 3$

$n/8 = \frac{n}{2^3} \longrightarrow 4$

$\vdots \qquad \vdots$

$1 = \frac{n}{2^K} \longrightarrow K+1$

#iterations $= K+1$

$= \log n + 1$

$\sim \log_2 n$. const

$\therefore O(\log_2 n)$

$\frac{n}{2^K} = 1$

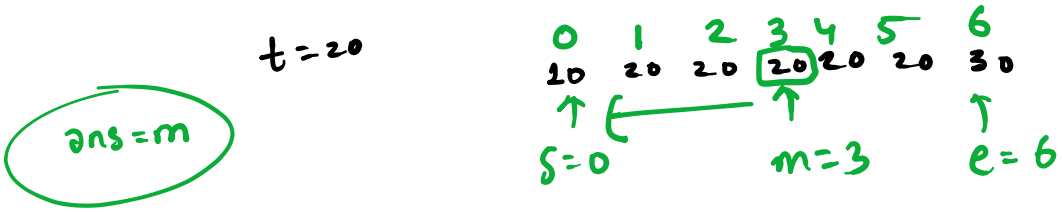
$2^K = n$

take $\log_2$ b/s

$K = \log_2 n$

$\log_2 2^K = K$

$\log_a a^b = b$



First Occurrence in Sorted Array

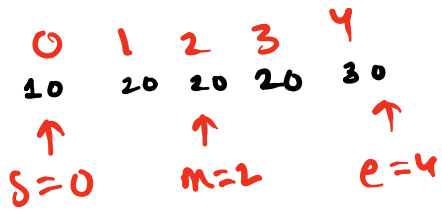
Given a **sorted** array of **N** integers, & an integer **T**, design an algorithm to find the **index of the 1st occurrence of T** in the given array. You've to output **-1** if **T** isn't present in the array.

t=20

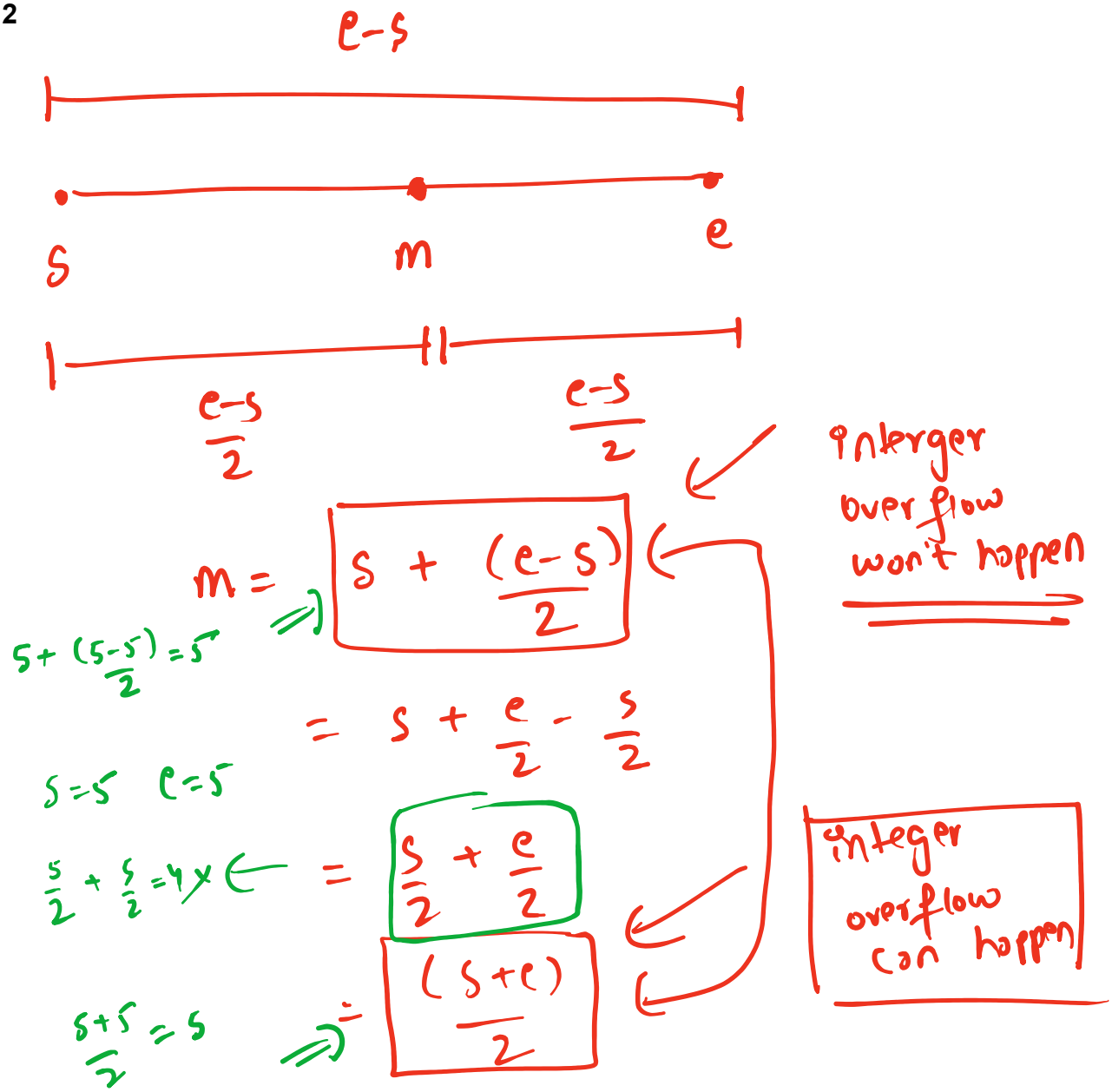
Example

Input : N = 8 ; T = 30

0	1	2	3	4	5	6	7
10	20	30	30	30	30	40	50



Output : 2



Last Occurrence in Sorted Array

Given a **sorted** array of **N** integers, & an integer **T**, design an algorithm to find the **index** of the **last occurrence** of **T** in the given array. You've to output **-1** if **T** isn't present in the array.

Example

Input : N = 8 ; T = 30

0	1	2	3	4	5	6	7
10	20	30	30	30	30	40	50

Output : 5

Count Occurrences in Sorted Array

Given a **sorted** array of **N** integers, & an integer **T**, design an algorithm to count the total no. of the **occurrences** of **T** in the given array. You've to output **-1** if **T** isn't present in the array.

Example

Input : N = 8 ; T = 30

0	1	2	3	4	5	6	7
10	20	30	30	30	30	40	50

Output : 4